
UNIX

Kompendium till 16-timmars UNIX-kurs

Kjell Post

Satt ur det ursprungliga föreläsningsmaterialet
(f11.tex-f14.tex, 1995)

21 september 2026

Om detta kompendium

Detta är föreläsningsmaterialet till en 16-timmars UNIX-kurs, ursprungligen skrivet som OH-bilder i $\text{\LaTeX}$ och här omsatt till kompendieform: en bild per avsnitt har blivit löpande text, bilderna har blivit avsnitt och underavsnitt, och innehållsförteckningen nedan gör materialet sökbart. Innehållet är oförändrat — det beskriver UNIX-världen som den såg ut när materialet skrevs, med SunOS 4.1.4, X11R5 och en Linuxkärna i version 1.2.

Kursen är indelad i fyra föreläsningar:

1. Introduktion, filsystemet, texteditorer och shellen.
2. AWK och shell-programmering.
3. UNIX-programmering i C: Standard I/O, systemanrop, `make`, `tar` och Internet-tjänster.
4. GNU-projektet, X-Windows, Tcl/Tk, Linux och UNIX-användarna.

Innehåll

Om detta kompendium	i
1 Introduktion, filsystem, editorer och shell	1
1.1 Introduktion	1
1.1.1 Vad är UNIX?	1
1.1.2 En session med UNIX	1
1.1.3 Principer bakom UNIX	2
1.1.4 Filosofi för UNIX-program	2
1.2 Kursplan	2
1.3 Bakgrund och historia	3
1.3.1 Litteraturförslag	3
1.3.2 Historia	3
1.3.3 Standardiseringar	3
1.4 Filsystemet	4
1.4.1 Olika typer av filer	4
1.4.2 Att ange filer	4
1.4.3 Vanliga filer	5
1.4.4 Kommandon för att manipulera filer	6
1.4.5 Manualsidor	7
1.4.6 Filsystemets internrepresentation	7
1.4.7 Länkar	8
1.4.8 Symboliska länkar	8
1.4.9 Filskydd och rättigheter	8
1.4.10 Att ändra rättigheter	9
1.5 Texteditorer	10
1.5.1 Emacs-terminologi	10
1.5.2 Emacs-trivia	11
1.6 Shellen	11
1.6.1 Översikt	12
1.6.2 UNIX-kommandon	12
1.6.3 Reguljära uttryck	12

1.6.4	Exempel	12
1.6.5	Andra specialtecken	13
1.6.6	Standard input, output och error	13
1.6.7	Exempel	14
1.6.8	Omdirigering av standard output	14
1.6.9	Omdirigering av standard input	15
1.6.10	Omdirigering av standard error	15
1.6.11	Pipes	15
1.6.12	Alias	16
1.6.13	Kommandoraden	16
1.6.14	Processhantering	17
1.6.15	Att slå ihjäl en process	17
1.6.16	För- och bakgrundsprocesser	18
1.6.17	Omgivningsvariabler	19
1.6.18	Shell-variabler	19
1.6.19	Initieringsfiler för C-shell	19
1.6.20	Programmet find	20
1.6.21	Reguljära uttryck i mönster	20
2	AWK och shell-programmering	23
2.1	AWK	23
2.1.1	Exempel	23
2.1.2	Strukturen i ett AWK-program	24
2.1.3	Hur AWK opererar	24
2.1.4	Användning	24
2.1.5	Egenskaper hos AWK	24
2.1.6	Mönstermatchning	25
2.1.7	Datavalidering	25
2.1.8	Variabler	25
2.1.9	Speciella variabler	25
2.1.10	Arrayer	26
2.1.11	Flerdimensionella arrayer	26
2.1.12	Godtyckliga array-index	26
2.1.13	Inbyggda funktioner	26
2.2	Shell-programmering	27
2.2.1	Exempel	27

2.2.2	Sökvägar	27
2.2.3	Argument på kommandoraden	28
2.2.4	Shift	28
2.2.5	Andra speciella variabler	28
2.2.6	If-then-else-satsen	29
2.2.7	Kommandot <code>test</code>	29
2.2.8	Case-satsen	29
2.2.9	For-satsen	30
2.2.10	While-satsen	30
2.2.11	Kommentarer	31
3	UNIX-programmering i C	33
3.1	C-program och Standard I/O	33
3.1.1	Argument på kommandoraden	33
3.1.2	Kommentarer	33
3.1.3	Standard I/O-biblioteket	34
3.1.4	Några vanliga Standard I/O-rutiner	34
3.1.5	Filpekare	35
3.1.6	Flaggor	36
3.1.7	Testa...	37
3.1.8	Läsa och skriva hela rader	37
3.1.9	Att läsa från en pipe	37
3.1.10	Att läsa omgivningsvariabler	38
3.2	Systemanrop	39
3.2.1	Lågnivå-I/O	39
3.2.2	Läsa och skriva	39
3.2.3	Skapa och öppna filer	40
3.2.4	Testa...	40
3.2.5	Kommentarer	41
3.2.6	I-noder	41
3.2.7	Kommentarer	42
3.2.8	Tidsangivelser	43
3.2.9	Processhantering	43
3.2.10	Systemanropet <code>fork</code>	43
3.2.11	Systemanropet <code>wait</code>	44
3.2.12	Typisk kod	44

3.2.13	Signaler och avbrottshantering	44
3.3	Kompilering med make	45
3.3.1	Programmet make	46
3.3.2	Exempel	46
3.3.3	Förenklingar	46
3.3.4	Omkompilering	47
3.3.5	Andra uppgifter	47
3.3.6	Variabler	47
3.3.7	Andra flaggor till make	48
3.4	Arkivfiler, tar -kommandot	48
3.5	Komprimerade filer, compress och gzip	48
3.5.1	Komprimerade tar-filer	49
3.6	Internet-tjänster	49
4	GNU, X-Windows, Tcl/Tk och Linux	51
4.1	GNU-projektet	51
4.1.1	Några GNU-program	51
4.1.2	Andra gratisprogram	52
4.2	X-Windows	52
4.2.1	Historia	52
4.2.2	Egenskaper	52
4.2.3	Server-client-koncept	52
4.2.4	X-Windows-programmering	53
4.3	Tcl/Tk	53
4.3.1	Kommentarer	53
4.4	Linux	54
4.4.1	Egenskaper	54
4.4.2	Vad behöver jag?	54
4.4.3	Portad programvara	54
4.5	UNIX-användare	55

Föreläsning 1

Introduktion, filsystem, editorer och shell

1.1 Introduktion

1.1.1 Vad är UNIX?

- **Ett operativsystem:**
 - Processhantering (multi-tasking/multi-user)
 - Filsystem
 - Resurshantering (minne, terminaler, ...)
 - Systemrutiner (t ex I/O)
 - Interface mot hårdvara
- **En utvecklingsmiljö för programmerare:**
 - *shell* — kommandotolken
 - Applikationer och utvecklingsverktyg

1.1.2 En session med UNIX

```
login: unix00
Password:
Last login: Sat Dec  9 13:11:39 from bilbo.mdh.se
SunOS Release 4.1.4 (GENERIC) #2: Fri Oct 14 11:09:47 PDT 1994

You have new mail.
bilbo% mail
Mail version SMI 4.0 Fri Oct 14 12:50:06 PDT 1994  Type ? for help.
"/usr/spool/mail/unix00": 1 message 1 new
>N  1 kpt@idt.mdh.se      Sun Dec 10 20:11   13/347   Hej
& 1
Message  1:
From: Kjell Post <kpt@idt.mdh.se>
To: unix00@idt.mdh.se
Subject: Hej

Välkommen till UNIX-kursen.

--Kjell
& d
& q
bilbo% who
kpt      tty2    Dec  4 00:07   (snorre.idt.mdh.s)
```

```
jma      ttyp3   Dec  6 10:21   (jma.idt.mdh.se:0)
root     ttyp4   Dec 10 19:44   (bilbo.mdh.se)
fsk      ttyp5   Dec  8 16:08   (fsk.idt.mdh.se)
bilbo% exit
Connection closed by foreign host.
```

1.1.3 Principer bakom UNIX

- Enkelt att porta, 99% av systemet är skrivet i C.
- Tillgängligt på många maskiner.
- Ett OS framförallt för programmerare

1.1.4 Filosofi för UNIX-program

- Små och enkla moduler.
- Löser en enstaka uppgift väl.
- "Pratar" inte i onödan.
- Goda möjligheter att "koppla ihop" program för speciella ändamål.
- Program skrivs i C och integreras väl med OS.

1.2 Kursplan

- **Introduktion:**
Litteratur; Historia; Standards
- **Filsystemet:**
Struktur; Kommandon
- **Texteditorer:**
Vanliga editorer; Emacs
- **Shellen:**
Kommandon; Processhantering; Miljövariabler; Utilities; Shell-programmering
- **Kärnan**
Systemanrop; Processer
- **Övrigt:**
Internet; X-Windows; GNU; Linux

1.3 Bakgrund och historia

1.3.1 Litteraturförslag

- **UNIX och X från början**
Torbjörn Andréasson, Jan Skansholm (Studentlitteratur)
- **The UNIX Programming Environment**
Brian Kernighan, Rob Pike (Prentice-Hall)
- **Advanced UNIX Programming**
Marc Rochkind (Prentice-Hall)
- **The Design of the UNIX Operating System**
Maurice Bach (Prentice-Hall)
- **The Design and Implementation of the 4.3BSD UNIX Operating System**
Samuel Leffler, et al (Addison-Wesley)

1.3.2 Historia

“The number of UNIX installations has grown to 10, with more expected.”

— The UNIX Programmer’s Manual (1972)

- **1969:** Bell Labs införskaffar en PDP-7.
- **1970:** Filsystemet skrivs, C-kompilatorn utvecklas.
Brian Kernighan myntar namnet “UNIX”
- **1973:** UNIX-kärnan skrivs om i C.
- **1974:** Börjar licensieras till universitet.
- **Idag:** Många varianter av UNIX
 - AT&T-baserade (System V): SunOS, Solaris
 - BSD (Berkeley Software Distribution)
 - OSF/1 UNIX, HP-UX, AIX, ...

1.3.3 Standardiseringar

- **POSIX:** Portable OS for Computer Environments.
Olika nivåer, t ex nivå 1: systemanrop & filsystem.
- **SVID:** System V Interface Definition
Stöder också POSIX.
- **X/Open:** Definitioner på kommandotolksnivån.

Alla UNIX-system är dock “moving targets”.

Vi kommer att använda SunOS (av Sun Microsystems).

1.4 Filsystemet

“Everything in the UNIX system is a file”

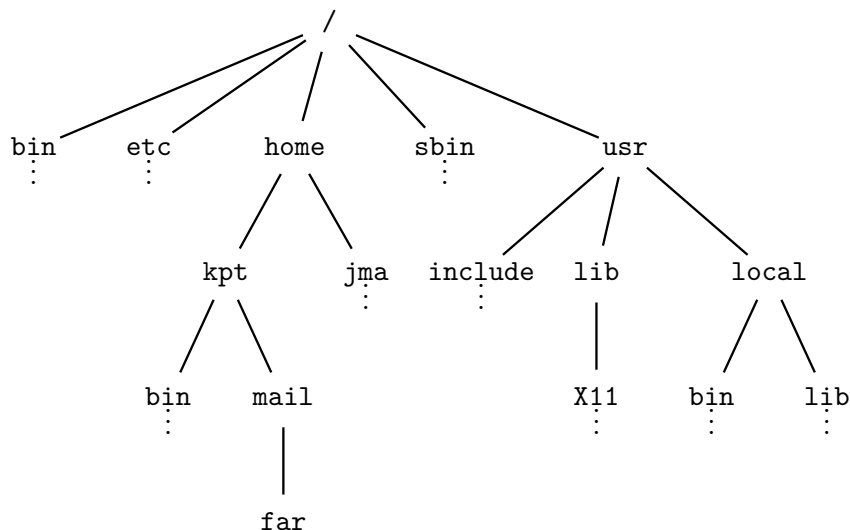
— Brian Kernighan

1.4.1 Olika typer av filer

- Vanliga filer (text, data, program, ...)
- Kataloger — innehåller namn på andra filer
- Speciella filer (tangentbord, skärm, disk, ...)

Filsystemet är organiserat som ett träd.

(Katalog heter på engelska *directory*.)



- Katalogen högst upp benämns *root*.
- Som användare befinner man sig alltid i en viss katalog, *current working directory*.

1.4.2 Att ange filer

- **Absolut:**

Symbolen "/" separerar namnen i sökvägen.

/home/kpt/mail/far	— ett brev till min far
/home/jma/	— Jukkas hemkatalog

- **Relativt:**

Sökvägen är relativ till *current working directory*.

<code>mail/far</code>	— brevet till far
<code>./mail/far</code>	— samma sak
<code>../jma/</code>	— Jukkas hemkatalog
<code>~jma</code>	— samma sak
<code>~kpt/mail/far</code>	— brevet till far
<code>~/mail/far</code>	— samma sak

- `.` är current working directory.
- `..` är föräldrakatalogen

Några viktiga filer och kataloger		
Sökväg	Betydelse	Innehåll
<code>/bin/</code> <code>/bin/cc</code>	<i>binaries</i>	Vanliga systemprogram C-kompilatorn
<code>/dev/</code> <code>/dev/null</code> <code>/dev/tty</code>	<i>devices</i> <i>null device</i> <i>teletype</i>	I/O-enheter Ett "svart hål" Tangentbordet
<code>/lib/</code> <code>/home/</code> <code>/usr/</code> <code>/usr/bin/</code> <code>/usr/include/</code> <code>/usr/local/</code>	<i>libraries</i> <i>user</i>	Biblioteksfiler Användarnas hemkataloger Andra filer Vanliga systemprogram <code>stdio.h</code> m.m. Lokala installationer
<code>/tmp/</code> <code>/sbin/</code>	<i>temporary</i>	Katalog för temporära filer Administrativa program

1.4.3 Vanliga filer

- Placeras i en katalog:
`/home/kpt/mail/far`
- Godtyckligt långa namn:
`Supercalifragilisticexpialidocius`
- Alla tecken utom `"/` kan förekomma:
`3a# "My File" .login`
Filer med konstiga tecken är svåra att hantera.
Filer som börjar med `."` syns normalt inte.
- Skillnad på gemener och versaler
`a.out` `A.out` — olika filer.
- Inga filtyper
Alla filer är en sekvens av bytes.

1.4.4 Kommandon för att manipulera filer

- `ls [options] [fil ...]` — visa filer

```
% ls /home/kpt/mail
far      sven      hans
% ls -a /usr/dict
./      ../      papers/  words
```

- `pwd` — print working directory

```
% pwd
/home/kpt
```

- `cd [dir]` — change directory

```
% cd mail
% pwd
/home/kpt/mail
% ls
far      sven      hans
```

- `mkdir dir` — make directory

```
% mkdir backup
% ls
backup/  far      sven      hans
```

- `rmdir dir` — remove directory

- `rm [file ...]` — remove file(s)

```
% rm hans
% ls
backup/  far      sven
% rm -i far
rm: remove far? n
% ls
backup/  far      sven
```

- `cp file1 file2` — copy files

```
% cp far backup/far
% ls backup
far
```

- `mv file1 file2` — move files

```
% mv far far.old
% mv backup bak
% ls
bak/      far.old  sven
```

- `cat [file ...]` — visa fil(er)

```
% cat sven
Tjena Sven, ring mig imorgon!
```

- `more [file ...]` — visa fil(er) sida för sida
- `file [file ...]` — gissa filtyp

```
% file sven
sven:      ascii text
```

- `echo [argument]` — eka argumentet

```
% echo hello world
hello world
```

1.4.5 Manualsidor

`man command` ger en manualsida för *command*

```
% man mkdir
NAME
    mkdir - make a directory
```

```
SYNOPSIS
    mkdir [ -p ] dirname...
```

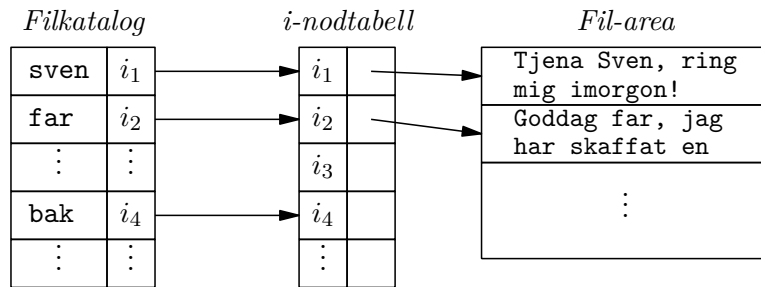
```
DESCRIPTION
    mkdir creates directories.  Standard entries,
    `.', for the directory itself, and `..' for
    its parent, are made automatically.
```

`man -k keyword` listar kommandon som berör *keyword*.

```
% man -k pizza
pizzatool (6) - a graphical pizza menu user interface
```

1.4.6 Filsystemets internrepresentation

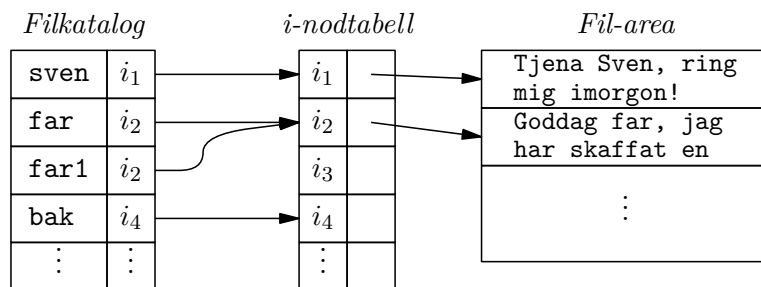
- En katalog är en lista av par (filnamn,i-node).
- En i-nod är ett heltal som identifierar
 - Filens innehåll
 - Filens datumstämpel
 - Filens skydd
 - ⋮



1.4.7 Länkar

Med länkar kan flera filnamn ha samma innehåll.

- `ln filename linkname` — make a link
- ```
% ln far far1
% ls -i far far1
31762 far 31762 far1
```



En fil tas aldrig bort förrän antalet länkar = 0.

```
% ls -l far far1
-rw-r--r- 2 kpt 493 Nov 30 15:27 far
% rm far
% ls -l far1
-rw-r--r- 1 kpt 493 Nov 30 15:27 far
```

### 1.4.8 Symboliska länkar

- `ln -s filename linkname` — make a symbolic link
- + En symbolisk länk kan sträcka sig över disk-partitioner.
- En symbolisk länk blir inaktuell om originalet tas bort.

### 1.4.9 Filskydd och rättigheter

Filer kan skyddas för tre kategorier av användare:

- **user:** Användaren själv

- **group:** Användare i samma grupp
- **others:** Alla andra

För varje kategori kan man specificera:

- **läsrättighet:** Anges med "r"
- **skriv rättighet:** Anges med "w"
- **exekveringsrättighet:** Anges med "x"

Tillsammans blir detta nio tecken:

| <i>user</i> |   |   | <i>group</i> |   |   | <i>others</i> |   |   |
|-------------|---|---|--------------|---|---|---------------|---|---|
| r           | w | x | r            | w | x | r             | w | x |

### Exempel:

```
% ls -l far1
-rw-r--r-- 1 kpt 493 Nov 30 15:27 far1
drwxr-xr-x 2 kpt 512 Dec 1 14:33 bak/
lrwxrwxrwx 1 kpt 3 Dec 1 14:34 bar -> foo
```

- `far1` kan läsas av alla men bara jag kan skriva.
- Filkataloger markeras med "d" till vänster.
- Symboliska länkar markeras med "l" till vänster.

#### 1.4.10 Att ändra rättigheter

- `chmod mode filename ...` — change mode  
*mode* är på formen *kategori +/- rättighet*.

### Exempel:

- Slå av läsrättigheter för alla andra:
 

```
% chmod go-r far1
% ls -l far1
-rw----- 1 kpt 493 Dec 1 14:40 far1
```
- Ändra så att alla kan skriva och läsa:
 

```
% chmod ugo+rw far1
% ls -l far1
-rw-rw-rw- 1 kpt 493 Dec 1 14:40 far1
```

## 1.5 Texteditorer

Några vanliga texteditorer i UNIX:

- **Emacs** — mycket kraftfull, programmerbar.  
I Emacs kan man kompilera, läsa mail, etc.
- **vi** — enkel; följer med alla UNIX-system.
- **textedit** — för SunOS; fönsterbaserad.

Emacs startas med kommandot `emacs filename`.

```
int main()
{
 printf("hello world\n");
}
```

---

```
---*-Emacs: test.c (C)--All-----
```

### 1.5.1 Emacs-terminologi

- **M-x** betyder tryck `ESC` sedan `x`.
- **C-c** betyder tryck `Ctrl` och `c`.

| Några vanliga Emacs-kommandon |                                |
|-------------------------------|--------------------------------|
| Kommando                      | Förklaring                     |
| C-x C-s                       | Spara filen                    |
| C-x C-w                       | Spara filen under annat namn   |
| C-x C-c                       | Avsluta                        |
| C-x C-z                       | Suspendera                     |
| C-f                           | Flytta framåt                  |
| C-b                           | Flytta bakåt                   |
| C-p                           | Flytta uppåt                   |
| C-n                           | Flytta nedåt                   |
| C-a                           | Flytta till början av raden    |
| C-e                           | Flytta till slutet av raden    |
| C-v                           | Flytta till nästa skärmsida    |
| M-v                           | Flytta till tidigare skärmsida |
| M-<                           | Flytta till början av filen    |
| M->                           | Flytta till slutet av filen    |

| Några vanliga Emacs-kommandon |                                |
|-------------------------------|--------------------------------|
| Kommando                      | Förklaring                     |
| <b>C-d</b>                    | Ta bort nästa tecken           |
| <b>C-k</b>                    | Ta bort resten av raden        |
| <b>C-_</b>                    | Undo                           |
| <b>C-s</b>                    | Sök framåt                     |
| <b>C-s C-s</b>                | Sök framåt efter samma text    |
| <b>C-r</b>                    | Sök bakåt                      |
| <b>C-r C-r</b>                | Sök bakåt efter samma text     |
| <b>C-g</b>                    | Avbryt ett kommando            |
| <b>C-space</b>                | Sätt märket                    |
| <b>C-w</b>                    | (Cut) Ta bort regionen         |
| <b>M-w</b>                    | (Copy) Spara regionen          |
| <b>C-y</b>                    | (Paste) Sätt in borttagen text |
| <b>M-x help-with-tutorial</b> |                                |

Regionen  $\equiv$  texten mellan markören och märket.

En vanlig kopieringsoperation är

- Gå till början av texten, sätt märket (**C-space**).
- Gå till slutet av texten.
- Kopiera regionen (**M-w**).
- Gå dit texten ska kopieras, tryck **C-y**.

### 1.5.2 Emacs-trivia

Emacs betyder "Editing Macros" och inte "Esc-Meta-Alt-Ctrl-Shift" eller "Eight Megabytes And Continuous Swapping" som många tror.

## 1.6 Shellen

Shellen är en programmerbar kommandotolk.

Shellen är ett helt vanligt C-program.

Vanliga implementationer:

- **sh** — Bourne shell; finns på alla UNIX-system.  
Används mest för scripts.
- **csh** — C-shell; en shell med C-liknande syntax.  
Var ganska länge den shell alla körde.

- **tcsch** — en **csch** med editering av kommandoraden.
- **ksh** — Korn Shell; förbättrad **sh**. Standard.
- **bash** — Bourne again shell; vanlig bland hackers.

Vi kommer att använda **csch** och dess syntax.

### 1.6.1 Översikt

| Egenskaper        | sh | csch | tcsch | ksh | bash |
|-------------------|----|------|-------|-----|------|
| kommando-historia |    | •    | •     | •   | •    |
| alias-funktion    |    | •    | •     | •   | •    |
| scripts           | •  | •    | •     | •   | •    |
| completion        |    | •    | •     | •   | •    |
| kommandoeditering |    |      | •     | •   | •    |
| job control       |    | •    | •     | •   | •    |

En användares shell definieras i filen `/etc/passwd`:

```

:
kpt:*:2007:10:Kjell Post:/home/kpt:/bin/csch

```

### 1.6.2 UNIX-kommandon

*kommando* [*-options*] [*argument*]

**Exempel:**

```

% rm -r -f bak
% rm -rf bak

```

### 1.6.3 Reguljära uttryck

UNIX erbjuder mer än vanliga wildcards:

```

? matchar exakt ett tecken.
* matchar godtyckligt många tecken.
[c1c2...cn] matchar något tecken $c \in \{c_1, c_2, \dots, c_n\}$.
[^c1c2...cn] matchar något tecken $c \notin \{c_1, c_2, \dots, c_n\}$.
[c1-cn] matchar något tecken i intervallet $c_1 \dots c_n$.

```

### 1.6.4 Exempel

```

% ls
Part0 Part1 Part2 part0 part1 part2
% ls part[0-2]
part0 part1 part2
% ls [pP]art[12]
Part1 Part2 part1 part2
% ls part[^1]
part0 part2

```

### 1.6.5 Andra specialtecken

- `#` — inleder kommentar till slutet av raden.

```
% echo hej # hopp
hej
```

- `;` — separerar flera kommandon på samma rad.

```
% cd .. ; pwd
```

- ``cmd`` — expanderas till resultatet av *cmd*.

```
% echo cwd = `pwd`
cwd = /home/kpt
```

- `\` — tolka inte nästa tecken.

```
% echo hej \# hopp
hej # hopp
```

- `'...'` — tolka inte tecknen inuti.

```
echo '# `;'
`;
```

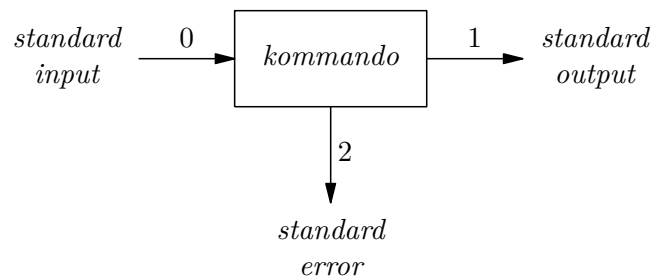
- `"..."` — expandera bara `$`, ``` och `!`

```
% echo "`pwd`"
/home/kpt
% echo '`pwd`'
`pwd`
```

Andra specialtecken behandlas senare.

### 1.6.6 Standard input, output och error

Många kommandon läser från *standard input* och skriver på *standard output*:



- *standard input* associeras vanligen med tangentbordet.
- *standard output* och *standard error* associeras vanligen med skärmen.

### 1.6.7 Exempel

```
% bc
1+2*3 Detta skriver jag på standard input
7 Detta får jag tillbaks på standard output
1+*2 Detta skriver jag på standard input
syntax error Skrivs på standard error

^d betyder "end-of-file"
%

% sort
1 Detta skriver jag på standard input
3
2
^d
1 Utskriften hamnar på standard output
2
3
```

**Exempel:** Om man inte vet i vilken ordning man ska klä på sig på morgonen kan man använda `tsort`:

```
% tsort
trousers shoes # byxorna före skorna
shirt tie
socks shoes
underwear trousers
underwear shoes
^d
shirt
tie
socks
underwear
trousers
shoes
```

### 1.6.8 Omdirigering av standard output

Med hjälp av `>` hamnar utskriften på en fil istället för på skärmen:

```
% cat sven
Tjena Sven, ring mig imorgon!
% cat sven > sven.bak
% cat sven.bak
Tjena Sven, ring mig imorgon!
```

Använder man `>>` adderas utskriften istället:

```
% cat >> sven.bak # läser från standard input
Hälsningar Kjell
% cat sven.bak
Tjena Sven, ring mig imorgon!
Hälsningar Kjell
%
```

### 1.6.9 Omdirigering av standard input

För att ett kommando ska läsa från en fil kan "<" användas:

```
% mail sm@prosolvias.se < sven.bak
% wc < sven.bak # räkna rader, ord, tecken
2 7 47
```

### 1.6.10 Omdirigering av standard error

Används mest då man vill spara felutskriften i en fil:

```
% cc hello.c >& errors
% cat errors
"hello.c", line 17: unknown size
```

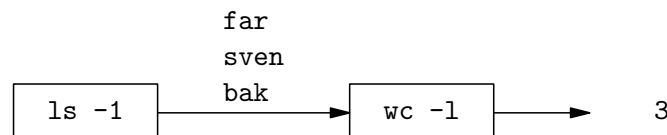
### 1.6.11 Pipes

Följande sekvens av kommandon är så vanlig

```
% ls -l > ls.out; wc -l < ls.out
3
```

att man skapat en speciell notation för ändmålet:

```
% ls -l | wc -l
3
```



Standard output för `ls -l`  $\equiv$  standard input för `wc -l`.

Några nya kommandon:

`grep mönster fil`  $\Rightarrow$  alla rader som matchar *mönster*.

`who`  $\Rightarrow$  vilka som är inloggade.

Dessa kan kopplas ihop med pipes för olika ändamål:

**Exempel:** Är Jukka inloggad?

```
% who | grep jma
```

**Exempel:** Hur många är inloggade?

```
% who | wc -l
```

**Exempel:** Hur många ggr är Jukka inloggad?

```
% who | grep jma | wc -l
```

### 1.6.12 Alias

Används för att definiera egna kommandon.

- `alias namn definition` — skapa alias  

```
% alias ll "ls -l"
```
- `alias namn` — visa alias  

```
% alias ll
ls -l
```
- `alias` — visa alla alias  

```
% alias
ll ls -l
rm rm -i
```
- `unalias namn` — ta bort alias  

```
% unalias ll
```

### 1.6.13 Kommandoraden

- Filnamnskomplettering:  

```
% set filec # slå på komplettering
% ls svesc
```

`esc` gör att C-shell fyller i resten av namnet, om inga tvetydigheter råder.
- Kommandohistoria:  

```
% history
264 ls sven
263 set filec
:
```
- Föregående kommando:  

```
% echo hej
hej
% !!
echo hej
hej
```
- Kommandouppprepning:  

```
% !264
ls sven
sven
%
```
- Kommandoeditering:  

```
% !264:s/sven/far
ls far
far
%
```

**Kommentarer:**

- `bash`, `tcsh`, m fl har annan, ofta trevligare syntax.
- `sh` har inga av dessa faciliteter.

**1.6.14 Processhantering**

En program som körs kallas en *process*.

UNIX är multitasking — flera processer kan köra samtidigt.

- `ps options`

— visa egna processer

```
% ps
 PID TT STAT TIME COMMAND
 1682 pb R 0:00 ps
 5133 pb S 0:05 -bash (bash)
```

| <i>Processinformation</i> |                                           |
|---------------------------|-------------------------------------------|
| PID                       | process-id                                |
| TT                        | associerad terminal                       |
| STAT                      | status: R=runnable, T=stopped, I=idle ... |
| TIME                      | förbrukad CPU-tid                         |
| COMMAND                   | Namn på kommandot                         |

**1.6.15 Att slå ihjäl en process**

```
kill [-signal] pid
```

Några vanliga signaler:

- `kill -15 pid`  
Ger processen en chans att terminera snyggt.  
Emacs sparar t ex sina filer.
- `kill -9 pid`  
Ovillkorligt – går inte att fånga.

`kill` borde egentligen heta `sendsignal` eftersom den inte bara används för terminering.

**Exempel:** Systemadmin vill slå ihjäl Jukkas processer:

```
ps aux|grep jma|awk '{print $2}'|xargs kill -9
```

### 1.6.16 För- och bakgrundsprocesser

Processer körs antingen i

- **Förgrunden:**

```
% command
```

Ingen ny prompt förrän *command* är klar.

- **Bakgrunden:**

```
% command &
```

```
[1] 276
```

```
%
```

Kommandot startas, process-id returneras och man får en ny prompt direkt.

```
% sleep 10; echo hej &
```

```
[1] 277
```

```
(10s senare)
```

```
%hej
```

Med följande kommandon kan man flytta processer från för- till bakgrunden och v v:

- **^z** — suspenderar förgrundsprocessen.

```
% sleep 10
```

```
^z
```

```
Stopped
```

```
%
```

- **bg** — lägger suspenderad process i bakgrunden.

```
% bg
```

```
[1] sleep 10 &
```

```
%
```

- **jobs** — Visa för- och bakgrundsjobb.

```
% jobs
```

```
[1] Running sleep 10
```

- **% *jobbnr*** — gör *jobbnr* till förgrundsprocess.

```
% %1
```

```
sleep 10
```

OBS: *jobbnr* (som visas med **jobs**) är inte samma som process-id!

### 1.6.17 Omgivningsvariabler

Arbetsomgivningen definieras av s k *environment variables*: Dessa konsulteras av diverse program.

- `echo $var` — visa en variabel

```
% echo $USER
kpt
```

- `setenv var value` — sätt en variabel

```
% setenv PRINTER laseridt
```

- `env` — visa alla variablers värden.

```
% env
TERM=vt100
PRINTER=laseridt
SHELL=/bin/csh
USER=kpt
EDITOR=emacs
```

### 1.6.18 Shell-variabler

Shell-variabler fungerar som omgivningsvariabler men ärvs inte av underprocesser.

- `set name=value` — sätt en shell-variabel

```
% set myname=kjell
% echo $myname
kjell
% csh # starta en ny csh
% echo $myname
myname: Undefined variable
% echo $PRINTER
laseridt
%
```

### 1.6.19 Initieringsfiler för C-shell

När man loggar in respektive startar en shell kommer visa filer i hemkatalogen att konsulteras.

- `~/.login`

Konsulteras då man loggar in.

- `~/.cshrc`

Konsulteras varje gång man startar `csh`.

I dessa filer kan man definiera alias, sätta variabler osv.

OBS: Alias ärvs inte av underprocesser så dessa måste definieras i `.cshrc`.

### 1.6.20 Programmet `find`

```
find sökväg uttryck
```

- Letar rekursivt i filkataloger efter något som matchar uttrycket.
- För varje träff kan ett kommando köras.

**Exempel:** Visa alla filer i denna katalog och nedåt:

```
% find . -print
./bak
./bak/far.bak
./far
```

**Exempel:** Leta efter filen `recept` i Jukkas hemkatalog:

```
% find ~jma -name "recept" -print
/home/jma/kakor/recept
```

```
grep options [-e]mönster fil ...
```

- Läser rader från *fil* eller standard input.
- Söker efter mönster i varje rad.
- Skriver resultat på standard output

**Exempel:** Hitta alla ord i ordlistan som innehåller två "v" efter varandra:

```
% grep ww /usr/dict/words
hollowware
screwworm
```

**Exempel:** Hitta alla filer vars namn innehåller "recept" med antingen versaler eller gemener:

```
% find . -print | grep -i recept
/home/kpt/vegmat/vegan.recept
```

### 1.6.21 Reguljära uttryck i mönster

Många UNIX-kommandon använder sig av reguljära uttryck för att specificera mönster.

| Mönster             | Betydelse                                      |
|---------------------|------------------------------------------------|
| <code>c</code>      | ett vanligt tecken                             |
| <code>\c</code>     | slå av betydelsen av <code>c</code>            |
| <code>^</code>      | början på raden                                |
| <code>\$</code>     | slutet på raden                                |
| <code>.</code>      | 1 tecken, vilket som helst                     |
| <code>[...]</code>  | vilket tecken som helst i intervallet          |
| <code>[^...]</code> | vilket tecken som helst, ej i intervallet      |
| <code>c*</code>     | noll eller flera förekomster av <code>c</code> |
| <code>c+</code>     | en eller flera förekomster av <code>c</code>   |

**Exempel:** Ord som kan skrivas på en räknedosa.

```
% grep '^[behilos]*$' /usr/dict/words | pr -135 -4
```

|        |         |        |       |
|--------|---------|--------|-------|
| b      | eh      | iii    | oil   |
| be     | el      | ill    | s     |
| bee    | ell     | is     | see   |
| bel    | else    | isle   | sell  |
| belie  | h       | l      | she   |
| belie  | he      | lee    | shell |
| bell   | heel    | less   | shill |
| belle  | hell    | lessee | shish |
| bib    | hellish | libel  | shoe  |
| bibb   | hello   | lie    | shoo  |
| bile   | hi      | lisle  | sib   |
| bill   | hill    | lo     | sill  |
| bless  | his     | lob    | silo  |
| bliss  | hiss    | lobe   | sis   |
| blob   | ho      | lobo   | slob  |
| bob    | hob     | loess  | sloe  |
| bobble | hobble  | loll   | slosh |
| boil   | hobo    | loose  | so    |
| bole   | hoe     | lose   | sob   |
| bolo   | hoi     | loss   | soil  |
| boo    | hole    | o      | sole  |
| boss   | hose    | obese  | solo  |
| e      | i       | oboe   |       |
| ebb    | ibis    | obsess |       |
| eel    | ii      | oh     |       |



## Föreläsning 2

# AWK och shell-programmering

## 2.1 AWK

Ett programmeringsspråk som lämpar sig för textmanipulering och generering av rapporter.

- `awk`, `nawk` — AT&T-versioner
- `gawk` — GNU-versionen av AWK.

### 2.1.1 Exempel

```
% cat names
Kjell Post 101687
Jan Gustafsson 101462
Filip Sebek 103113
% awk '{print "Namn:",$1,$2,"Telefon:",$3}' names
Namn: Kjell Post Telefon: 101687
Namn: Jan Gustafsson Telefon: 101462
Namn: Filip Sebek Telefon: 103113
```

### Typiska arbetsuppgifter:

- **Urval:** Skriv ut alla rader med mer än 72 tecken.

```
% awk 'length>72' report.txt
```

- **Arrangering:** Skriv ut 3:e fältet, sedan 2:a.

```
% echo 10 20 30 | awk '{ print $3,$2 }'
30 20
```

- **Rapporter:** Addera alla fälten i kolumn 1, skriv sedan ut summan och medelvärde.

```
% awk '{ sum+=$1} END { print sum,sum/NR }'
10 apelsiner
20 bananer
5 kantareller
70 lingon
^d
105 26.25
```

### 2.1.2 Strukturen i ett AWK-program

```
mönster { kod }
mönster { kod }
⋮
```

- *mönster* är ett reguljärt, numeriskt eller sträng-uttryck, eller en kombination av dessa.
- *kod* är exekverbar kod, med C-liknande syntax.

### 2.1.3 Hur AWK opererar

För varje indata-rad:

För varje mönster:

Om mönster matchar raden:

Kör koden.

**Exempel:**     `length > 72 { print }`

Om det finns en mönster utan kod, skriv ut raden som matchar:

```
length > 72
```

Om det finns kod utan tillhörande mönster, kör koden på varje rad:

```
{ print $3, $2 }
```

### 2.1.4 Användning

```
awk 'program' [fil1 fil2...]
```

```
awk -fprogfil [fil1 fil2...]
```

```
awk 'length > 72' in1 in2
```

### 2.1.5 Egenskaper hos AWK

- Strängar omvandlas till heltal "automagiskt".
- Arrayer med godtyckliga index.
- Indata läses automatiskt.
- Rader delas upp i fält (`$1 ... $NF`) automatiskt.
- Kontrollsatser som i C: `if-then-else`, `while`, ...
- Inbyggda funktioner för strängar, reguljära uttryck, m m. Användardefinierade funktioner.
- `printf` för utskrift, `getline` för inmatning.  
Till/från godtyckliga filer eller processer.

### 2.1.6 Mönstermatchning

Några exempel på mönster:

|                          |                                   |
|--------------------------|-----------------------------------|
| <code>/^\$/</code>       | matchar en blank rad.             |
| <code>/text/</code>      | om raden inkluderar <i>text</i> . |
| <code>/[0-9]+\$</code>   | om raden slutar med ett heltal.   |
| <code>/^[A-Za-z]/</code> | om raden börjar med en bokstav.   |
| <code>!/.../</code>      | om raden inte matchar "..."       |

### 2.1.7 Datavalidering

```
% cat /etc/passwd
ken:oLS4spqrnwylw:6:1:m0130,m322:/usr/ken:
dmr:35qxyzz/C5thc:7:4:mhc699,m069:/usr/dmr:
bwk:95Lz1pIkpuurA:9:1:mh1548,m301:/usr/bwk:

passwd - check password file
BEGIN { FS = ":" }
NF != 7 {
 printf("line %d does not have 7 fields\n", NR) }
$1 ~ /^[A-Za-z0-9]/ {
 printf("line %d, nonalpha user id\n", NR) }
$2 == "" {
 printf("line %d, no password\n", NR) }
$3 ~ /^[0-9]/ {
 printf("line %d, nonnumeric user id\n", NR) }
$4 ~ /^[0-9]/ {
 printf("line %d, nonnumeric group id\n", NR) }
$6 !~ /\^\/ {
 printf("line %d, invalid login dir\n", NR) }
```

### 2.1.8 Variabler

Variabler är typlösa.

```
diam = "100"; pi = 3.14; yta = pi*diam
```

### 2.1.9 Speciella variabler

|                       |                                                                |
|-----------------------|----------------------------------------------------------------|
| <code>\$n</code>      | <i>n</i> -te fältet på raden ( <code>\$0</code> är hela raden) |
| <code>NF</code>       | antalet fält                                                   |
| <code>NR</code>       | antalet lästa rader                                            |
| <code>FS</code>       | fältseparatorn (default: blank eller tab)                      |
| <code>OFS</code>      | utskriftsseparatorn (blank)                                    |
| <code>FILENAME</code> | namn på aktuell fil                                            |

### 2.1.10 Arrayer

**Exempel:** Reversera en fil:

```
{ x[NR] = $0 }
END { for (i = NR; i > 0; i--)
 print x[i] }
```

### 2.1.11 Flerdimensionella arrayer

```
x[row, col] = ...
```

Samma som `x[row " ," col] = ...`

### 2.1.12 Godtyckliga array-index

| Indata  | Utdata  |
|---------|---------|
| ATT 100 | IBM 900 |
| IBM 100 | ATT 400 |
| DEC 100 | DEC 400 |
| HP 100  | HP 100  |
| ATT 50  |         |
| ATT 200 |         |
| IBM 200 |         |
| DEC 100 |         |
| IBM 500 |         |
| ATT 50  |         |

```
{ amount[$1] += $2 }
END { for (name in amount)
 print name, amount[name] \
 | "sort +1 -nr" }
```

### 2.1.13 Inbyggda funktioner

- `getline` — läser nästa indata-rad.

I `nawk` kan man också skriva t ex

```
"ps aux" | getline x
```

- `index( $s_1, s_2$ )` — ger positionen för  $s_2$  i  $s_1$ .
- `length( $s$ )` — returnerar antal tecken i  $s$ .
- `substr( $s, m, n$ )` — ger tecken  $m \dots n$  av  $s$ .
- `system( $cmd$ )` — anropar shellen med  $cmd$ .

Många fler inbyggda funktioner finns – se manualen!

*"AWK is one of the tools that you hardly notice when you use it, but miss terribly when you have to use a system without it."* — M. Banahan & A. Ritter

## 2.2 Shell-programmering

Shellen är också ett programmeringsspråk.

Skapa ett shell-script:

1. Skapa filen: `emacs filnamn`.
2. Slå på exekveringsrättigheter: `chmod +x filnamn`
3. Kör scriptet: `filnamn`

Första raden i scriptet definierar vilken shell som används:

- `#!/bin/sh` Bourne-shell (vanligast för script)
- `#!/bin/csh` C-shell

osv; vi kommer att använda Bourne-shell för scripts.

### 2.2.1 Exempel

```
% cat info
#!/bin/sh
display name, working directory, date, time
echo "Username : `whoami`"
echo "Directory : `pwd`"
echo "Date & Time: `date`"

% info
Username : kpt
Directory : /home/kpt/kurser
Date & Time: Tue Dec 5 15:44:27 MET 1995
```

### 2.2.2 Sökvägar

Antag att jag skapar ett eget program `ls`.

Hur kan shellen veta vilken `ls` som ska köras?

```
% cat > ls
#!/bin/sh
echo surprise
^d
% chmod +x ls
% ls
bak/ far sven
% ./ls
surprise
%
```

Variabeln `PATH` anger i vilka kataloger shellen ska söka efter kommandon.

```
% echo $PATH
/bin:/usr/bin:/usr/local/bin:/home/kpt/bin:.
```

Kommandot `which` anger vilket kommando som utförs:

```
% which ls
/bin/ls
```

### 2.2.3 Argument på kommandoraden

|                          |                  |
|--------------------------|------------------|
| <code>\$0</code>         | Programmets namn |
| <code>\$1 ... \$9</code> | Argument 1 ... 9 |
| <code>\$*</code>         | Alla argument    |
| <code>\$#</code>         | Antal argument   |

```
#!/bin/sh
echo '$0 =' $0
echo '$1 =' $1 '$2 =' $2 '$3 =' $3
echo '$* =' $*
echo '$# =' $#
```

```
% testarg fie foo fum
$0 = testarg
$1 = fie $2 = foo $3 = fum
$* = fie foo fum
$# = 3
```

### 2.2.4 Shift

Kommandot `shift` gör att  $\$(n + 1)$  blir  $\$n$ :

```
#!/bin/sh
echo $1 $2 $3
shift
echo $1 $2 $3

% testshift a b c d
a b c
b c d
```

### 2.2.5 Andra speciella variabler

|                          |                                                                                                              |
|--------------------------|--------------------------------------------------------------------------------------------------------------|
| <code>\$?</code>         | Exit-status för förra kommandot                                                                              |
| <code>\$\$</code>        | Process-id för detta kommando                                                                                |
| <code>\$var</code>       | Värdet av variabeln <i>var</i>                                                                               |
| <code>\${var}</code>     | Som ovan (krävs om t ex <code>_</code> ingår i <i>var</i> )                                                  |
| <code>\${var-val}</code> | <code>\${var}</code> om <i>var</i> definierad, annars <i>val</i> .                                           |
| <code>\${var=val}</code> | <code>\${var}</code> om <i>var</i> definierad, annars <i>val</i> samt att <i>var</i> sätts till <i>val</i> . |
| <code>\${var?msg}</code> | <code>\${var}</code> om <i>var</i> definierad, annars skriv ut <i>msg</i> och avbryt exekveringen.           |
| <code>\${var+msg}</code> | <i>val</i> om <i>var</i> definierad, annars inget.                                                           |

### 2.2.6 If-then-else-satsen

```

 if test
then kommandon
elif test
then kommandon
:
else kommandon
fi

```

```

#!/bin/sh
here user
user=$1
if who | grep -s $user
then echo "$user is logged in"
else echo "$user is not logged in"
fi

% here jma
jma is not logged in

```

### 2.2.7 Kommandot test

```

if test ! -f $FILE
then ...

```

|                                                            |                                                                        |
|------------------------------------------------------------|------------------------------------------------------------------------|
| <code>! <i>test</i></code>                                 | sant om <i>test</i> är falskt                                          |
| <code>-e <i>fil</i></code>                                 | sant om <i>fil</i> existerar.                                          |
| <code>-d <i>fil</i></code>                                 | sant om <i>fil</i> är en katalog.                                      |
| <code>-f <i>fil</i></code>                                 | sant om <i>fil</i> är en vanlig fil.                                   |
| <code>-r <i>fil</i></code>                                 | sant om <i>fil</i> är läsbar.                                          |
| <code>-w <i>fil</i></code>                                 | sant om <i>fil</i> är skrivbar.                                        |
| <code>-x <i>fil</i></code>                                 | sant om <i>fil</i> är exekverbar.                                      |
| <code>-z <i>s</i></code>                                   | sant om längden på strängen <i>s</i> är noll.                          |
| <code>-n <i>s</i></code>                                   | sant om <i>s</i> inte är tom.                                          |
| <code><i>s</i><sub>1</sub> = <i>s</i><sub>2</sub></code>   | sant om <i>s</i> <sub>1</sub> = <i>s</i> <sub>2</sub> .                |
| <code><i>s</i><sub>1</sub> != <i>s</i><sub>2</sub></code>  | sant om <i>s</i> <sub>1</sub> ≠ <i>s</i> <sub>2</sub> .                |
| <code><i>n</i><sub>1</sub> -eq <i>n</i><sub>2</sub></code> | sant om talen <i>n</i> <sub>1</sub> och <i>n</i> <sub>2</sub> är lika. |

man test ger många andra testvillkor.

### 2.2.8 Case-satsen

```

 case str in
mönster) kommandon ;;
:
mönster) kommandon ;;
esac

```

```
#!/bin/sh
case $# in
 0) echo "usage: print file ..."; exit 1 ;;
) a2ps $ | lpr ;;
esac
```

Kommentar: `a2ps` konverterar en textfil till PostScript; `lpr` skickar en fil till skrivaren.

### 2.2.9 For-satsen

```
 for var in lista
do
kommandon
done
```

```
#!/bin/sh
compare files in two directories
case $# in
 [^2]) echo "usage: cmpdir dir1 dir2"; exit 1 ;;
esac
for i in `ls $1`
do
 echo $i:
 cmp $1/$i $2/$i
done
```

### 2.2.10 While-satsen

```
 while kommandon
do
kommandon
done
```

```
#!/bin/sh
case $# in
 [^1]) echo "usage: waitfor user"; exit 1 ;;
esac
while true
do
 if who | grep $1
 then exit
 fi
 sleep 60
done
```

**Uppgift:** Skriv ett kommando `zap` *namn* som interaktivt frågar om du vill döda alla processer som matchar *namn*.

**Exempel:**

```
% zap sleep
552 pp1 S 0:00 sleep 100? y
555 pp1 S 0:00 egrep sleep? n
```

Börja med att se vad **ps** genererar:

```
% ps -ag | grep $*
595 pp1 S 0:00 sleep 100
597 pp1 S 0:00 egrep sleep
% ps -ag | grep $* | awk '{print $1}'
595
598

#!/bin/sh
PATH=/bin:/usr/bin

case $1 in
 "") echo 'usage: zap [-sig] pattern' 1>&2; exit 1
 ;;
 -*) SIG=$1; shift
 esac

ps -ag | egrep $* | while read line
do
 echo -n "$line? " > /dev/tty
 read response < /dev/tty
 case $response in
 y*) kill $SIG `echo $line | awk '{print $1}'`
 ;;
 q*) break
 esac
done
```

### 2.2.11 Kommentarer

- **PATH** sätts explicit så att inte andra **ps**, **egrep**, **awk**, etc, ska synas.
- **1>&2** är Bourne-shellens syntax för att dirigera om *standard output* (1) till *standard error* (2).
- Vi har lagt till möjligheten att kunna ange signal, t ex **zap -9 namn**. Observera hur **shift** används för att **\$1** sedan ska innehålla processnamnet.
- Vi använder **egrep** istf **grep** så att man kan skriva t ex **zap 'sleep|date'**.
- När vi frågar användaren skriver vi på terminalen (**/dev/tty**) så att inte utskriften försvinner ner i en pipe.
- Inläsning sker med **read**, också från terminalen, eftersom *standard input* är kopplat till utdata från **ps**.



## Föreläsning 3

# UNIX-programmering i C

### 3.1 C-program och Standard I/O

Varför programmera i C när det finns script?

- Hantera icke text-filer.  
En del filer, t ex `/usr/utmp`, är i binärformat.
- Robusthet.  
Ett C-program har bättre kontroll över resurser, t ex minnet.
- Effektivitet.  
Men kom ihåg att det oftast går snabbare att skriva ett script!

#### 3.1.1 Argument på kommandoraden

```
#include <stdio.h>

int main(int argc, char *argv[])
{
 int i;
 printf("argc = %d\n", argc);
 for (i = 0; i < argc; i++)
 printf("argv[%d] = %s\n", i, argv[i]);
 exit(0);
}

% gcc main.c
% a.out foo bar fum
argc = 4
argv[0] = a.out
argv[1] = foo
argv[2] = bar
argv[3] = fum
```

#### 3.1.2 Kommentarer

- Ett C-program kompileras normalt till filen `a.out`.  
Använd flaggan `-o` om ni vill ha ett annat namn:

```
% gcc -o main main.c
```

- `argv` är en array av strängar.
- `argv[0]` är namnet på programmet.

- De ”användbara” argumenten är

```
argv[0] ... argv[argc-1]
```

- Den vanliga C-kompilatorn `cc` klarar inte ANSI-C så vi använder GNU:s kompilator `gcc`.
- Anropet `exit(0)` garanterar ”normal” exit-status.

### 3.1.3 Standard I/O-biblioteket

En samling rutiner för effektiv och portabel I/O.

- `/usr/include/stdio.h`:  
Innehåller prototypdeklarationer och makros.

```
#include <stdio.h>
```

- `/usr/lib/libc.a`:  
Innehåller separatkompilerade rutiner.

`man stdio` beskriver biblioteket.

Standard I/O finns också på andra plattformar.

### 3.1.4 Några vanliga Standard I/O-rutiner

- `int getchar()`  
Hämtar nästa tecken från *standard input*.  
Returnerar EOF när det är slut på indata.
- `putchar(int c)`  
Skriver ut `c` på *standard output*.
- `printf(char *format, arg,...)`  
Formatterad utskrift, t ex  

```
printf("%s: can't open %s\n",
 argv[0], argv[1]);
```
- `int scanf(char *format, pekare, ...)`  
För inläsning och konvertering till strängar, tal,...  

```
scanf("%d", &x);
```

**Exempel:** Skriv ett program `kat` i stil med `cat` som kopierar *standard input* till *standard output*.

```
/* kat -- copy stdin to stdout */

#include <stdio.h>

main()
{
```

```
 int c;

 while ((c = getchar()) != EOF)
 putchar(c);
 exit(0);
}
```

- `c` är en `int` så att `EOF` får plats.
- Programmet fungerar som `cat` utan argument.

**Exempel:** Utöka programmet så det klarar filargument:

```
% copy fil1 fil2 ...
```

Nu räcker det inte att läsa från *standard input*, vi måste vi kunna öppna filer, osv.

- `FILE *fopen(char *name, char *mode)`  
där `mode = "r"` för läsning, `mode = "w"` för skrivning, osv.
- `fclose(FILE *fp)`
- `int getc(FILE *fp)`
- `putc(int c, FILE *fp)`
- `fprintf(FILE *fp, format, arg, ...)`
- `fscanf(FILE *fp, format, pekare, ...)`

`fp` kallas en *filpekare*.

### 3.1.5 Filpekare

En filpekare pekar på en struktur som innehåller bl a

- teckenbuffert
- aktuell position i filen
- läs- eller skriv-status
- $\vdots$

Tre fördefinierade filpekare finns alltid:

```
FILE *stdin;
FILE *stdout;
FILE *stderr;
```

```

/* kat [file ...] -- copy file(s) to standard output */

#include <stdio.h>

void copy(FILE *fp)
{
 int c;

 while ((c = getc(fp)) != EOF)
 putchar(c);
}

main(int argc, char *argv[])
{
 int i;
 FILE *fp;

 if (argc == 1)
 copy(stdin);
 else
 for (i = 1; i < argc; i++)
 if ((fp = fopen(argv[i], "r")) == NULL) {
 fprintf(stderr, "%s: can't open %s\n",
 argv[0], argv[i]);
 exit(1);
 } else {
 copy(fp);
 fclose(fp);
 }
 exit(0);
}

```

### 3.1.6 Flaggor

**Exempel:** Utöka programmet så specialtecken skrivs ut oktalt om flaggan `-v` anges:

```
% kat -v fil1 fil2 ...
```

Ändra först `copy`:

```

void copy(FILE *fp)
{
 int c;
 while ((c = getc(fp)) != EOF)
 if (optv==0 || isprint(c) || isspace(c))
 putchar(c);
 else
 printf("\\%03o", c);
}

```

Variabeln `optv` är sann (`=1`) om `-bv` har angetts.

```
int optv=0; /* 1 => make ctrl chars visible */
```

I `main` kollar vi först vilka flaggor som angetts:

```

while (argc > 1 && argv[1][0] == '-') {
 switch (argv[1][1]) {
 case 'v':
 optv = 1;
 break;
 default:
 fprintf(stderr,
 "%s: unknown argument %s\n",
 argv[0], argv[1]);
 exit(1);
 }
 argc--;
 argv++;
}

```

### 3.1.7 Testa...

```

% kat -v
^A^Bhej^F^G
\001\002hej\006\007

```

### 3.1.8 Läs och skriv hela rader

- `int fgets(char *buf, int size, FILE *fp)`  
läser `size-1` tecken och lägger dessa i `buf`, följt av ett `NULL`-tecken. `fgets` returnerar
  - `NULL` vid EOF.
  - `buf` annars.

Om raden är för lång måste du kontrollera det.

- `fputs(char *buf, FILE *fp)`  
skriver ut strängen `buf` på filen `fp`.
- `fflush(FILE *fp)`  
tvingar ut all buffrad information till filen.

### 3.1.9 Att läsa från en pipe

- `FILE *popen(char *cmd, char *mode)`  
Fungerar som `fopen` men `cmd` är ett kommando.

**Exempel:** Kommandot `zap namn`, nu skrivet i C.

```

#include <stdio.h>
#include <signal.h>

int ttyin()
{
 char buf[BUFSIZ];
 static FILE *tty = NULL;

 if (tty == NULL)

```

```

 tty = fopen("/dev/tty", "r");
 if (tty == NULL)
 fprintf(stderr, "can't happen: no /dev/tty\n"), exit(1);
 if (fgets(buf, sizeof buf, tty) == NULL ||
 buf[0] == 'q')
 exit(0);
 else
 return buf[0];
}

main(int argc, char *argv[]) /* zap name */
{
 FILE *fin;
 char buf[BUFSIZ];
 int pid;

 if ((fin = popen("ps -ag", "r")) == NULL)
 fprintf(stderr, "%s: can't run ps -ag\n",
 argv[0]), exit(1);
 fgets(buf, sizeof buf, fin); /* header */
 fprintf(stderr, "%s", buf);
 while (fgets(buf, sizeof buf, fin) != NULL)
 if (argc == 1 || strstr(buf, argv[1])) {
 buf[strlen(buf)-1] = '\0'; /* kill \n */
 fprintf(stderr, "%s? ", buf);
 if (ttyin() == 'y') {
 sscanf(buf, "%d", &pid);
 kill(pid, SIGKILL);
 }
 }
 exit(0);
}

```

- `strstr` letar efter delsträngar.
- `kill` skickar en signal till en process.
- `SIGKILL` är definierad i `<signal.h>`.

### 3.1.10 Att läsa omgivningsvariabler

- `char *getenv(char *name)`

Returnerar en shell-variabels värde som en sträng, eller `NULL` om variabeln inte är definierad.

```

#include <stdio.h>
#include <stdlib.h> /* för getenv */

main()
{
 char *home = getenv("HOME");
 printf("Your home directory: %s\n", home);
 exit(0);
}

% gcc envtest.c
% a.out
Your home directory is: /home/idt/kpt

```

## 3.2 Systemanrop

Ett gränssnitt mot operativsystemet.

- In- och utmatning
- Filsystemet
- Processer
- Signaler och avbrottshantering.

Systemanropen är dokumenterade i sektion 2 av manualen, t ex

```
% man 2 open
```

De flesta systemanrop returnerar `-1` vid fel.

### 3.2.1 Lågnivå-I/O

All I/O i UNIX sker genom att läsa/skriva filer.

Istället för filpekare används *fildeskriptor*, ett heltal.

- En filpekare innehåller bl a en fildeskriptor.
- En fildeskriptor fås t ex genom att öppna eller skapa en fil.
- Ett program ärver alltid tre fördefinierade fildeskriptorer:
  - 0 – *standard input*.
  - 1 – *standard output*.
  - 2 – *standard error*.

### 3.2.2 Läsa och skriva

- `int read(int fd, char *buf, int n)`

Läser max `n` bytes från `fd` till `buf` och returnerar

- `-1` om ett fel inträffade.
  - 0 om vi kommit till EOF.
  - antalet lästa bytes, vilket kan vara  $< n$  om det inte fanns mer att läsa.
- `int write(int fd, char *buf, int n)`

Skriver ut `n` bytes från `buf` till `fd` och returnerar också antalet överförda bytes, eller `-1` eller 0.

Ett vanligt värde för `n` är `BUFSIZ` (från `stdio.h`).

**Exempel:** Minimal version av `kat`:

```
#include <stdio.h> /* för BUFSIZ */

main()
{
 char buf[BUFSIZ];
 int n;

 while ((n = read(0, buf, sizeof buf)) > 0)
 write(1, buf, n);
 exit(0);
}
```

Att läsa eller skriva 1 byte i taget går också bra (kärnan buffrar data) men systemanrop tar tid.

### 3.2.3 Skapa och öppna filer

- `int open(char *name, int rwmode)`  
Returnerar en fildeskriptor eller `-1` vid fel.  
`rwmode` är antingen `0` (läsa), `1` (skriva) eller `2` (läsa och skriva).
- `int creat(char *name, int perms)`  
Skapar en ny fil med skyddskoder satt enligt `perms`, ett oktalt tal (se `man chmod`).
- `int close(int fd)`  
Stänger en fil. Görs automatiskt vid `exit`.
- `int unlink(char *path)`  
Tar bort en fil från filsystemet.

**Exempel:** Minimal version av `cp`:

```
#include <stdio.h>
#define USAGE "usage: %s from to"

main(int argc, char *argv[])
{
 int f1, f2, n;
 char buf[BUFSIZ];
 if (argc != 3)
 fprintf(stderr, USAGE, argv[0]), exit(1);
 if ((f1 = open(argv[1], 0)) == -1)
 perror(argv[1]), exit(1);
 if ((f2 = creat(argv[2], 0644)) == -1)
 perror(argv[2]), exit(1);
 while ((n = read(f1, buf, sizeof buf)) > 0)
 if (write(f2, buf, n) != n)
 perror(0), exit(1);
 exit(0);
}
```

### 3.2.4 Testa...

```
% a.out foo bar
% ls -l bar
```

```
-rw-r--r-- 1 kpt 530 Dec 14 10:30 bar
% a.out urk bar
urk: No such file or directory
```

### 3.2.5 Kommentarer

- `perror(s)` skriver ut `s` följt av ett meddelande om senaste systemanropsfelet.
- Programmet bevarar inte skyddskoderna utan sätter kopian till 0644 vilket betyder `rw-r--r--`.
- Programmet klarar inte att kopiera flera filer till ett katalognamn.

### 3.2.6 I-noder

För att få information om en fil måste man först...

```
#include <sys/types.h>
#include <sys/stat.h>
```

och sedan använda någon av

- `int stat(char *path, struct stat *buf)`  
Första argumentet är t ex `/usr/dict/words`.
- `int lstat(char *path, struct stat *buf)`  
Som `stat` men följer inte symboliska länkar.
- `int fstat(int fd, struct stat *buf)`  
Första argumentet är en fildeskriptor.

`struct stat` är definierad i `<sys/stat.h>` och innehåller följande fält:

```
dev_t st_dev; /* device file resides on */
ino_t st_ino; /* the file serial number */
mode_t st_mode; /* file mode */
nlink_t st_nlink; /* number of hard links to the file */
uid_t st_uid; /* user ID of owner */
gid_t st_gid; /* group ID of owner */
dev_t st_rdev; /* device identifier (special files only)*/
off_t st_size; /* total size of file, in bytes */
time_t st_atime; /* file last access time */
time_t st_mtime; /* file last modify time */
time_t st_ctime; /* file last status change time */
long st_blksize; /* preferred blocksize for file system I/O*/
long st_blocks; /* actual number of blocks allocated */
```

Makros för att testa `st_mode` för olika filtyper:

```
S_ISLNK(m) =1 för en symbolisk länk.
S_ISDIR(m) =1 för en filkatalog.
S_ISREG(m) =1 för en vanlig fil.
:
:
```

Bitmaskar för att testa `st_mode`:

```
st_mode & S_IRUSR ≠ 0 om user får läsa.
st_mode & S_IWUSR ≠ 0 om user får skriva.
st_mode & S_IXUSR ≠ 0 om user får exekvera.
st_mode & S_IRGRP ≠ 0 om group får läsa.
st_mode & S_IWGRP ≠ 0 om group får skriva.
st_mode & S_IXGRP ≠ 0 om group får exekvera.
st_mode & S_IROTH ≠ 0 om other får läsa.
st_mode & S_IWOTH ≠ 0 om other får skriva.
st_mode & S_IXOTH ≠ 0 om other får exekvera.
```

**Exempel:** Bevaka brevlådan för inkommande post.

```
/* checkmail -- watch user's mailbox */

#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>

main(int argc, char *argv[])
{
 struct stat buf;
 char *name, *getlogin();
 int lastsize = 0;

 if ((name = getlogin()) == NULL)
 fprintf(stderr, "you don't exist\n"), exit(1);
 if (chdir("/usr/spool/mail") == -1)
 fprintf(stderr, "no mail directory\n"), exit(1);
 for (;;) {
 if (stat(name, &buf) == -1) /* no mailbox */
 buf.st_size = 0;
 if (buf.st_size > lastsize)
 fprintf(stderr, "\nYou have new mail\007\n");
 lastsize = buf.st_size;
 sleep(60);
 }
}
```

### 3.2.7 Kommentarer

- `getlogin()` returnerar användarens namn, t ex `kpt`.
- Breven ligger lagrade i `/usr/spool/mail/namn`.
- `chdir` förflyttar processen till en annan katalog.
- Fältet `st_size` anger filens storlek.
- `sleep(60)` gör att programmet förbrukar ca en CPU-sekund per timme.

### 3.2.8 Tidsangivelser

Fältet `st_mtime` anger när filen skapades.

Tiden anges i sekunder sedan 1 jan 1970.

**Exempel:** Ta reda på när en fil skapades:

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>

main(int argc, char *argv[])
{
 struct stat buf;
 if (argc == 2 && stat(argv[1], &buf) != 1)
 printf("%s\n", ctime(&buf.st_mtime));
 exit(0);
}
```

`ctime` konverterar till `tex` Thu Dec 14 12:37:20 1995.

### 3.2.9 Processhantering

Hur kan man starta ett annat program från ett befintligt?

Tre sätt:

- `system(char *cmd)`

```
char buf[80];
sprintf(buf, "lpr %s\n", filename);
system(buf);
```

- `execvp(char *filename, char *argv[])`

Startar ett program utan att komma tillbaka.

`filename` är namnet på programmet som ska köras; `argv` är `argv` för nya programmet.

- `fork + exec` — se nästa sida.

#### 3.2.10 Systemanropet `fork`

- `int fork()`

`fork` klyver processen i två kopior, båda fortsätter exekveringen.

I den ena kopian (barnet) returnerar `fork` 0.

I den andra kopian (föräldern) returnerar `fork` processnumret för barnet.

```
int pid;
switch (pid = fork()) {
 case -1:
 perror(progname);
 exit(1);
 case 0:
 /* I'm the child */
 default:
 /* I'm the parent */
}
```

### 3.2.11 Systemanropet wait

- `int wait(int *statusp)`

Väntar tills någon barnprocess terminerat.

Returnerar process-id för barnet.

`statusp` innehåller information om hur barnet terminerade (se `man 2 wait`).

### 3.2.12 Typisk kod

```
int status;

if (fork() == 0)
 execvp(...); /* child */
wait(&status); /* parent */
```

I princip fungerar `system()` på detta vis.

### 3.2.13 Signaler och avbrottshantering

Ibland vill man fånga signaler, t ex förhindra att användar trycker ctrl-C:

```
#include <signal.h>

signal(SIGINT, SIG_IGN)
```

Signalen `SIGINT` genereras av ctrl-C och `SIG_IGN` gör att den ignoreras.

```
signal(SIGINT, SIG_DFL)
```

Detta återställer behandlingen av `SIGINT`.

Man kan också sätta upp en hanterare (funktion) som körs då en viss signal fångas.

**Exempel:** `timeout [-s] kommando`.

Kör ett kommando. Om kommandot inte har exekverat klart på `s` sekunder slås det ihjäl. Default är 10 sekunders tidsfrist.

```
% timeout -5 sleep 10
sleep killed
% timeout -10 sleep 5
%
```

- Kommandot körs som en barnprocess mha `fork`.
- Föräldern deklarerar sedan en signalhanterare så att funktionen `onalarm` anropas vid `SIGALRM`. Den dödar barnet om tidsfristen är ute.
- `alarm(sec)` genererar `SIGALRM` efter `sec` sekunder.
- Barnets `exit`-status returneras också av `timeout`-kommandot (se `man 2 wait`).

```

#include <stdio.h>
#include <signal.h>

int pid; /* child process id */

main(int argc, char *argv[])
{
 int sec = 10, status, onalarm();

 if (argc > 1 && argv[1][0] == '-') {
 sec = atoi(&argv[1][1]);
 argc--;
 argv++;
 }
 if (argc < 2)
 fprintf(stderr, "usage: %s [-10] command", argv[0]), exit(1);
 if ((pid = fork()) == 0) {
 execvp(argv[1], &argv[1]);
 fprintf(stderr, "couldn't start %s\n", argv[1]), exit(1);
 }
 signal(SIGALRM, onalarm);
 alarm(sec);
 if (wait(&status) == -1 || (status & 0177) != 0)
 fprintf(stderr, "%s killed\n", argv[1]), exit(1);
 exit((status >> 8) & 0377);
}

onalarm() /* kill child when alarm arrives */
{
 kill(pid, SIGKILL);
}

```

### 3.3 Kompilering med make

Antag att källkoden existerar i flera filer:

|                    |
|--------------------|
| main.c             |
| /* main.c */       |
| #include "utils.h" |
| ⋮                  |

|                    |
|--------------------|
| utils.c            |
| /* utils.c */      |
| #include "utils.h" |
| ⋮                  |

|               |
|---------------|
| utils.h       |
| /* utils.h */ |
| #define N 10  |
| ⋮             |

Hur ska man kompilera? Olika alternativ:

1. Allt på en gång:

```
% gcc -o pgm utils.c main.c
```

2. Separatkompilering för hand:

```

% gcc -c main.c
% gcc -c utils.c
% gcc -o pgm main.o utils.o

```

Flaggan `-c` producerar en *objektfil* (`.o`-fil). Sista kommandot länkar `.o`-filerna till en exekverbar fil.

3. Använda en `makefile` (visas snart):

```
% make
gcc -c main.c
gcc -c utils.c
gcc -o pgm main.o utils.o
```

### 3.3.1 Programmet `make`

Ett program för att specificera kompileringsförfarandet.

- `make` [*flaggor*] [*makro-definitioner*] [*objekt*]

När kommandot ges utan argument hämtas instruktionerna från filen `makefile` eller `Makefile`.

Makefilen innehåller regler som beskriver hur filer ett projekt beror av varandra och vilka kommandon som måste köras för att uppdatera systemet.

`make` tittar på filernas tidsstämpel för att avgöra vilka kommandon som ska köras.

Flaggan `-f` specificerar en annan fil:

```
% make -f rules
```

### 3.3.2 Exempel

```
CC=gcc

pgm: main.o utils.o
 gcc -o pgm main.o utils.o

main.o: main.c utils.h
 gcc -c main.c

utils.o: utils.h
 gcc -c utils.c
```

- `make` har variabler för val av t ex C-kompilator (`CC`).
- Rad 3,6,9 innehåller *beroenden*. Rad 3 säger t ex att `pgm` beror på `main.o` och `utils.o`.
- Rad 4,7,10 innehåller kommandon och måste börja med en `tab`!

### 3.3.3 Förenklingar

`make` har en del "inbyggd intelligens", den vet t ex hur `.o` filer ska produceras ur `.c`-filer.

```
CC=gcc
pgm: main.o utils.o
 gcc -o pgm main.o utils.o
```

Testa:

```
% make
gcc -target sun4 -c main.c
gcc -target sun4 -c utils.c
gcc -o pgm main.o utils.o
```

### 3.3.4 Omkompilering

Prova att ändra bara en fil och se vad `make` gör:

```
% touch utils.c
% make
gcc -target sun4 -c utils.c
gcc -o pgm main.o utils.o
```

Kommandot `touch` sätter bara filens datumpstämpel så att den verkar sprillans ny.

Som synes utför `make` bara de kompileringar som behövs.

### 3.3.5 Andra uppgifter

Man kan också definiera objekt som utför annat än kompilering:

```
CC=gcc
pgm: main.o utils.o
 gcc -o pgm main.o utils.o

clean:
 rm -rf pgm main.o utils.o
```

Testa:

```
% make clean
rm -rf pgm main.o utils.o
%
```

### 3.3.6 Variabler

Man kan deklarera och använda variabler i Makefilen:

```
CC=gcc
OBJS=main.o utils.o

pgm: $(OBJS)
 gcc -o pgm $(OBJS)

clean:
 rm -rf pgm $(OBJS)
```

Eller sätta variabler från kommandoraden:

```
% make CFLAGS=-g # generate debug info
gcc -g -target sun4 -c utils.c
gcc -o pgm main.o utils.o
```

### 3.3.7 Andra flaggor till `make`

- `make -f filnamn`  
läser regler från *filnamn* istf `[mM]akefile`.
- `make -n`  
skriver ut vad som ska göras men utför det inte.
- `make -s`  
(silent) skriver inte ut vad som görs.

Andra versioner av `make` har andra flaggor, se `man make`.

## 3.4 Arkivfiler, `tar`-kommandot

- `tar flaggor [tar-fil] [fil ...]`

Några vanliga flaggor till `tar`:

- `tar -cf arkiv.tar filer` (c = create)  
Skapar `arkiv.tar` med *filer* som innehåll.
- `tar -tvf arkiv.tar` (t = table of contents)  
Visar vad som finns i `arkiv.tar` utan att packa upp.
- `tar -xf arkiv.tar` (x = extract)  
Packar upp `arkiv.tar`. Var filerna hamnar beror på hur tar-filen skapades och var du står.  
Använd `tvf` först!

## 3.5 Komprimerade filer, `compress` och `gzip`

De två vanligaste komprimeringsprogrammen är

- `compress`  
En compressad fil har vanligtvis filsuffix `.Z`.

|                                      |                                          |
|--------------------------------------|------------------------------------------|
| Komprimering:                        | <code>compress <i>filnamn</i></code>     |
| Packa upp:                           | <code>uncompress <i>filnamn.z</i></code> |
| Packa upp till <code>stdout</code> : | <code>zcat <i>filnamn.z</i></code>       |

- `gzip`  
En gzippad fil har vanligtvis filsuffix `.gz`.

|                                      |                                         |
|--------------------------------------|-----------------------------------------|
| Komprimering:                        | <code>gzip <i>filnamn</i></code>        |
| Packa upp:                           | <code>gunzip <i>filnamn.gz</i></code>   |
| Packa upp till <code>stdout</code> : | <code>gzip -cd <i>filnamn.gz</i></code> |

### 3.5.1 Komprimerade tar-filer

tar-filer på Internet är oftast komprimerade med `compress` eller `gzip`.

- `filnamn.tar.z` — en compressad tar-fil.  
Uppackning: `zcat filnamn.tar.z | tar -xf -`  
Flaggan `-` gör att `tar` läser från `stdin`.
- `filnamn.tar.gz`, eller
- `filnamn.tgz` — en gzippad tar-fil  
Uppackning: `gzip -cd filnamn.tgz | tar -xf -`

Ibland kan det vara bra att se var filen packas upp:

```
% gzip -cd arkiv.tgz | tar -tvf -
```

## 3.6 Internet-tjänster

En snabb översikt av UNIX-kommandon för nättjänster:

- `talk` — interaktivt "samtalsprogram"

```
% talk harding@am.ucsc.edu
[Waiting for your party to respond]
Hi Kjell! How are you?
Ok.
```

```

Fine, thanks. Say, how about lunch?
Great, see you there. :-)
<ctrl-c>
```

- Elektronisk mail

```
% mail unix10@bilbo.mdh.se
Subject: Uppgifterna
Ni är klara med laborationerna!

--Kjell
<ctrl-d>
```

Mer sofistikerade mail-klienter finns givetvis.

- `ftp` — filöverföring. Anonym FTP:

```
% ftp ftp.sunet.se
Connected to ftp.sunet.se.
220 ftp.sunet.se FTP server ready.
Name (ftp.sunet.se:kpt): ftp
331 Guest login ok, send your complete
 e-mail address as password.
Password: -kpt@mdh.se
```

```
ftp> cd pub/gnu
ftp> ls -l "|grep less.*tar"
less-290.tar.gz
ftp> bin
200 Type set to I.
ftp> get less-290.tar.gz
168349 bytes received in 3.1 seconds
ftp> quit
%
```

- `rlogin`, `telnet` — fjärrinloggning

```
% rlogin am.ucsc.edu -l kepost
Password:
am% logout
Connection closed
%
```

- `nslookup` — slå upp IP-nummer

```
% /usr/etc/nslookup bilbo.mdh.se
Server: lennart.mdh.se
Address: 130.238.210.15

Name: bilbo.mdh.se
Address: 130.238.250.126
```

- `ping` — skicka testpaket

Fullständig sökväg: `/usr/etc/ping`.

Används för att se om en annan dator är ”uppe”:

```
% ping snorre.idt.mdh.se
snorre.idt.mdh.se
% ping elvis
elvis is alive
```

- Dessutom: Netscape, WAIS, gopher, archie, ...

## Föreläsning 4

# GNU, X-Windows, Tcl/Tk och Linux

### 4.1 GNU-projektet

GNU-projektet drivs av *Free Software Foundation*.

Initiativtagare till FSF är Richard Stallman (`rms`), författare av Emacs, GCC och många andra GNU-program. GNU = "GNU's Not UNIX"

Free Software — gratis, men inte helt utan restriktioner:

- Källkoden är tillgänglig från många FTP-sites.
- Du kan modifiera koden som du vill men:
  - Om du distribuerar programmet måste du erbjuda källkoden också.
  - Du får inte sälja GNU-programvara, men du kan ta betalt för distribution och underhåll.

FSF's copyright-princip kallas *copyleft*.

#### 4.1.1 Några GNU-program

Ersätter/förbättrar existerande UNIX-applikationer.

- `emacs`
- `gcc`, `g++` — C/C++-kompilator.
- `gawk`, `gnutar`, `gnumake`, ... — UNIX-verktyg.
- `gzip` — förbättrat compress-verktyg.
- `bison` — ersättare för `yacc`.
- `flex` — ersättare för `lex`.
- `bash` — Bourne again shell.
- `ghostscript/ghostview` — PostScript-viewers.
- `rcs` — versionshantering.

GNU-programvara finns speglad på bl a `ftp.sunet.se` och `ftp.isy.liu.se` under `pub/gnu`.

Original-siten är `prep.ai.mit.edu` i USA.

De flesta filerna är packade med `gzip`.

### 4.1.2 Andra gratisprogram

- T<sub>E</sub>X och L<sup>A</sup>T<sub>E</sub>X

Typsättning av dokument. Exempel:

$$\$G_{\{in_{\{j\}}\}} = G_{\{\infty_{\{j\}}\}} \backslash \tanh (\backslash ell_{\{j\}} / \backslash lambda_{\{j\}}) \$$$

$$\Rightarrow G_{in_j} = G_{\infty_j} \tanh(\ell_j / \lambda_j)$$

- X-Windows

Ett fönstersystem för UNIX-maskiner.

- Linux

En UNIX-implementation för 386/486-maskiner.

## 4.2 X-Windows

### 4.2.1 Historia

- **1980** — MIT och DEC startar ett projekt ”W”.
- **1986** — Nytt projekt  $\Rightarrow$  X-windows 10.4.
- **1987** — X11R1
- **1991** — X11R5
- **1995** — X11R6

### 4.2.2 Egenskaper

- Väldigt generell. Portad till olika plattformar.
- Kan skräddarsys för olika ändamål.
- OpenWindows, Motif m fl använder X11.

### 4.2.3 Server-client-koncept

X-Windows är nätverksorienterat: ett program kan köra på en maskin och visa resultat på ett annat.

- **Server:** Programmet som sköter displayen (fönster, osv). Kör vanligtvis på en lokal arbetsstation.
  - Tolkar meddelanden från olika klienter.
  - Förmedlar användarens aktioner till klienten.
  - Gör 2D grafik.
  - Upprätthåller fönster, typsnitt, etc.
- **Klient:** Applikationen som antingen körs lokalt eller på en ”större” maskin.  
Exempel: **xterm** – terminalemulator; **xman** – manualläsare; ...

### 4.2.4 X-Windows-programmering

Att programmera GUIs i ”ren” X11 är mycket omständigt.

Det finns dock paket som underlättar, t ex `libsx`:

```
#include "libsx.h"

main()
{
 MakeLabel("Hello World");
 MainLoop();
}

% gcc -o hello hello.c \
 /usr/lib/libsx/src/libsx.a \
 -lXaw -lXmu -lXt -lX11 -lXext
% hello
```

## 4.3 Tcl/Tk

Ett annat system för att skriva GUI i X11-miljö.

- **Tcl:** Ett script-språk med strängar, loopar, funktioner, etc, samt möjlighet att anropa C-kod.
- **Tk:** En samling funktioner för att skapa knappar, fönster, formulär, etc.

**Exempel:**

```
#!/usr/local/bin/wish -f
label .l -text "Filename:"
entry .e -relief sunken -width 30 \
 -textvariable fname
pack .l -side left
pack .e -side left -padx 1m -pady 1m
bind .e <Return> { exec emacs $fname }
```

### 4.3.1 Kommentarer

- `wish` är kommandotolken för Tcl-scripts.
- `label` skapar en knapp `l` med texten `Filename:`.
- `entry` skapar ett formulär `e`. Text som matas in hamnar i variabeln `fname`.
- `pack` packar ihop knappen och formuläret i ett enda fönster.
- `bind` knyter ett kommando, `emacs $fname` till formuläret – när någon trycker return där kommer kommandot att köras.

## 4.4 Linux

Linux är en fri UNIX för 386, Alpha, Amiga, Atari, ...

Ursprungligen utvecklad av Linus Torvalds, Helsinki.

Hundratusentals användare över hela världen.

Källkoden kan hämtas från bl a

|                              |                           |
|------------------------------|---------------------------|
| <code>ftp.funet.fi</code>    | <code>pub/OS/Linux</code> |
| <code>sunsite.unc.edu</code> | <code>pub/Linux</code>    |
| <code>tsx-11.mit.edu</code>  | <code>pub/linux</code>    |

Linux förekommer i olika s k distributioner.

Slackware är en populär sådan.

### 4.4.1 Egenskaper

- Källkoden tillgänglig. Ingen "hemlig" AT&T-kod.
- POSIX-kompatibel.

### 4.4.2 Vad behöver jag?

- En PC med ISA, EISA, PCI eller VLB-buss.
- Minimum: 386SX/16, 2 MB RAM, 1.44 MB eller 1.2 MB floppy.
- Bekvämt: 12MB RAM och 300MB hårddisk om du vill ha köra X-Windows, Emacs och annat godis.
- Specifika hårdvarukrav finns i **Hardware-HOWTO** på FTP-arkiven.

### 4.4.3 Portad programvara

- **UNIX-kommandon:** `ls`, `awk`, ...
- **Utvecklingsverktyg:** `gcc`, `gdb`, `make`, `bison`, `flex`, `rcs`, ...
- **Språk:** C/C++, Objective-C, Modula-3, Ada, Pascal, FORTRAN, ML, Scheme, Tcl/Tk, Perl, Common Lisp, ...
- **Fönstersystem:** X11R5, X11R6
- **Editors:** `emacs`, `vi`, ...
- **Shells:** `bash`, `csch`, `tcsh`, ...
- **Kommunikation:** SLIP, PPP, UUCP, Kermit, ...
- **News/Mail:** `nn`, `rn`, `pine`, ...
- **Typsättning:**  $\text{\LaTeX}$ , `groff`, ...

## 4.5 UNIX-användare

- **Nyborjare:** Har fortfarande problem att komma ihåg `<RETURN>` efter varje rad.
- **Novis:** Har haft sin första dåliga `rm`-erfarenhet.
- **Användare:** Har hört talas om reguljära uttryck men aldrig sett ett.
- **Kunnig användare:** Använde `sed` en gång men kollade resultatet efteråt.
- **Expert:** Skriver C-program med Emacs och kompilerar med `cc`. Vet vad `&&` och `||` betyder.
- **Hacker:** Skriver C-program med `cat >` och kompilerar med `!cc`.
- **Guru:** Skriver assembler-kod med `cat >`. Patchar kärnan när flera användare är inloggade. Läser device driver-listningar till frukost. Använder `make` för mer än två kommandon.
- **Wizard:** Skriver device-drivers med `cat >`. Fixar buggar genom att patcha binärfiler i Emacs. Kan svara på godtycklig UNIX-fråga innan du ställer den.